BỘ CÔNG THƯƠNG

TRƯỜNG ĐẠI HỌC CÔNG THƯƠNG TP. HCM

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO TIẾN ĐỘ NHÓM 2

MÔN HỌC: DỮ LIỆU LỚN (BIG DATA)

ĐỀ TÀI: XÂY DỰNG HỆ THỐNG GỢI Ý PHIM TRỰC TUYẾN DỰA TRÊN KIẾN TRÚC LAKEHOUSE VÀ ĐIỆN TOÁN PHÂN TÁN

Họ và tên	**Mã số sinh viên**
Phan Trần Hữu Duy	2001230128
Chung Tuấn Hỷ	2001230342
Đặng Thái Hoà	2001230254

Giảng viên: Huỳnh Thái Học

TP. HỒ CHÍ MINH, THÁNG 10 NĂM 2026

DANH SÁCH THÀNH VIÊN NHÓM VÀ BẢNG PHÂN CÔNG NHIỆM VỤ

Họ và tên	Mã số sinh viên	Vai trò	Nhiệm vụ chính phụ trách	Tỷ lệ đóng góp
Phan Trần Hữu Duy	2001230128	Nhóm trưởng	- Thực hiện các cài đặt trên máy cá nhân và hướng dẫn các thành viên khác	**34%**
Chung Tuấn Hỷ	2001230342	Thành viên	- Dựa vào hướng dẫn và cài đặt, thay đổi theo cấu trúc phù hợp trên máy cá nhân	**33%**
Đặng Thái Hoà	2001230254	Thành viên	- Dựa vào hướng dẫn và cài đặt, thay đổi theo cấu trúc phù hợp trên máy cá nhân	**33%**

MỤC LỤC

CHƯƠNG I. TỔNG QUAN ĐỀ TÀI 6
1. Mục tiêu và bài toán của đồ án 6
2. Nguồn dữ liệu, cấu trúc dữ liệu và quy mô dữ liệu 8
3. Môi trường thực nghiệm 11
CHƯƠNG II. KIẾN TRÚC HỆ THỐNG VÀ CÔNG NGHỆ SỬ DỤNG 12
1. Kiến trúc Medallion Lakehouse 12
2. Bảng tổng hợp công nghệ sử dụng và vai trò 13
3. Đánh giá tính chất điện toán phân tán của hệ thống 14
CHƯƠNG III. TIẾN ĐỘ THỰC HIỆN VÀ MINH CHỨNG 15
1. Giai đoạn 1: Chuẩn bị môi trường và cụm HDFS 2 DataNode 15
2. Giai đoạn 2: Đưa dữ liệu thô lên tầng Bronze 16
3. Giai đoạn 3: Đọc và xử lý ghép nối dữ liệu bằng PySpark 17
4. Giai đoạn 4: Làm sạch dữ liệu và ghi định dạng Parquet tầng Silver 19
5. Giai đoạn 5: Truy vấn phân tích bằng Spark SQL 22
6. Giai đoạn 6: Huấn luyện mô hình gợi ý ALS bằng Spark MLlib 24
7. Giai đoạn 7: Sinh gợi ý và lưu trữ vào tầng Gold 27
8. Giai đoạn 8: Thu nhận dữ liệu thời gian thực bằng Kafka & Spark Streaming 28
9. Giai đoạn 9: Tự động hóa điều phối bằng Apache Airflow 31
10. Giai đoạn 10: Xây dựng giao diện phục vụ dữ liệu FastAPI (Hoàn thành bản thử nghiệm) 33
CHƯƠNG IV. TỔNG HỢP TIẾN ĐỘ, KẾT QUẢ, KHÓ KHĂN VÀ KẾ HOẠCH TIẾP THEO 35
1. Bảng tổng hợp tiến độ các hạng mục 35
2. Kết quả đạt được và so sánh hiện trạng hệ thống 36
3. Khó khăn gặp phải và giải pháp kỹ thuật đã xử lý 36
4. Các phần chưa hoàn thành, hạn chế kỹ thuật và kế hoạch tiếp tục 37
KẾT LUẬN 39

DANH MỤC HÌNH ẢNH

Hình 2.1. Sơ đồ kiến trúc Medallion Lakehouse của hệ thống 12
Hình 3.1. Giao diện quản trị NameNode HDFS (2 Live Nodes) 16
Hình 3.2. Kiểm tra các tiến trình phân tán Hadoop bằng lệnh jps 16
Hình 3.3. Danh sách tệp dữ liệu thô trên tầng Bronze HDFS 17
Hình 3.4. Kết quả ghép (join) dữ liệu phim bằng PySpark 18
Hình 3.5. Sơ đồ DAG thực thi trên giao diện Spark UI 19
Hình 3.6. Quá trình làm sạch và tối ưu dữ liệu tầng Silver 22
Hình 3.7. Cấu trúc tệp Parquet tầng Silver lưu trữ trên HDFS 22
Hình 3.8. Kết quả truy vấn phân tích dữ liệu bằng Spark SQL 23
Hình 3.9. Kết quả huấn luyện mô hình ALS và đánh giá RMSE 26
Hình 3.10. Dữ liệu gợi ý phim tầng Gold lưu trữ trên HDFS 28
Hình 3.11. Hoạt động phát và nhận dữ liệu thời gian thực qua Kafka 30
Hình 3.12. Log streaming được lưu trữ trên HDFS 31
Hình 3.13. Giao diện điều phối luồng công việc Apache Airflow 33
Hình 3.14. Giao diện Web Dashboard FastAPI hiển thị dữ liệu các tầng 34

DANH MỤC BẢNG BIỂU

Bảng 1. Cấu trúc chi tiết (Schema) của các tệp dữ liệu trong hệ thống 8
Bảng 2. Phân bổ tệp và dung lượng lưu trữ theo các tầng Medallion trên HDFS 10
Bảng 3. Bảng tổng hợp công nghệ, phiên bản và vai trò trong hệ thống 13
Bảng 4. Siêu tham số và chỉ số huấn luyện mô hình ALS 25
Bảng 5. Một số kết quả gợi ý mẫu lưu ở tầng Gold 27
Bảng 6. Tổng hợp tiến độ các hạng mục thực hiện đến thời điểm báo cáo.....35
Bảng 7. So sánh hiện trạng hệ thống trước và sau giai đoạn tối ưu hóa 36
Bảng 8. Kế hoạch chi tiết triển khai giai đoạn 2 hoàn thiện đồ án.....38

CHƯƠNG I. TỔNG QUAN ĐỀ TÀI

1. Mục tiêu và bài toán của đồ án

1.1. Mục tiêu đề tài

Đề tài hướng tới việc nghiên cứu, làm chủ và ứng dụng các công nghệ Dữ liệu lớn (Big Data) hiện đại nhằm giải quyết bài toán gợi ý phim cá nhân hóa cho người dùng trong nền tảng phim trực tuyến. Các mục tiêu cụ thể gồm:

1. Xây dựng kho dữ liệu phân tán theo kiến trúc Medallion Lakehouse (Bronze → Silver → Gold) trên nền tảng Apache Hadoop HDFS với cấu hình mô phỏng phân tán gồm 2 DataNode độc lập.
2. Thiết kế và vận hành quy trình xử lý dữ liệu theo lô (Batch Processing) bằng Apache Spark (PySpark): đọc dữ liệu thô, loại bỏ dữ liệu bất thường/rác, chuẩn hóa lược đồ dữ liệu và nén lưu trữ tối ưu dưới định dạng cột Parquet (Snappy).
3. Ứng dụng thư viện học máy phân tán Spark MLlib, huấn luyện mô hình lọc cộng tác ALS (Alternating Least Squares) trên quy mô hàng triệu bản ghi đánh giá, tự động dự đoán sở thích và sinh Top 5 phim gợi ý tối ưu nhất cho từng người dùng lưu trữ ở tầng Gold.
4. Triển khai đường ống thu nhận dữ liệu thời gian thực (Streaming Ingestion) kết hợp giữa Message Broker Apache Kafka (vận hành chế độ KRaft) và Spark Structured Streaming nhằm hứng liên tục luồng log tương tác (click, xem, đánh giá) của người dùng và lưu trữ bền vững xuống HDFS.
5. Thiết lập cơ chế tự động hóa quy trình (Workflow Orchestration) bằng Apache Airflow, điều phối lịch trình chạy tự động hàng ngày cho các tác vụ xử lý theo lô.
6. Xây dựng tầng phục vụ dữ liệu (Serving Layer) bằng FastAPI Dashboard kết nối trực tiếp với HDFS qua giao diện WebHDFS REST API để trực quan hóa dữ liệu các tầng và cung cấp kết quả gợi ý.

1.2. Phát biểu bài toán

- **Đầu vào (Input):**
  - *Dữ liệu tĩnh lịch sử:* Gồm 25 triệu lượt đánh giá (ratings) và 1 triệu từ khóa gắn thẻ (tags) của 162.541 người dùng trên 62.423 bộ phim từ tập dữ liệu MovieLens 25M, kết hợp thông tin chi tiết (tóm tắt, điểm số TMDb, kinh phí, ngày phát hành) từ The Movies Dataset (Kaggle).
  - *Dữ liệu động thời gian thực:* Luồng bản tin sự kiện dạng JSON mô phỏng các hành động tức thời của người dùng (click, view, add_to_watchlist, rate) phát liên tục qua Kafka broker.

- **Quy trình xử lý (Processing):**
  - Trích xuất, làm sạch, giải quyết xung đột khóa/tên cột, chuyển đổi định dạng và nén lưu trữ theo 3 tầng Medallion.
  - Phân tích thống kê bằng Spark SQL và phân rã ma trận tương quan người dùng - phim bằng thuật toán học máy phân tán ALS.
  - Hứng luồng dữ liệu micro-batch streaming ghi xuống HDFS theo thời gian thực.
- **Đầu ra (Output):**
  - Kho lưu trữ Lakehouse chuẩn hóa tối ưu dung lượng và tốc độ truy vấn trên cụm phân tán HDFS.
  - Danh sách gợi ý Top 5 bộ phim có điểm số dự đoán cao nhất cho từng người dùng lưu tại tầng Gold.
  - Giao diện Web API phục vụ tra cứu dữ liệu danh mục phim và hiển thị bảng gợi ý theo thời gian thực.

1.3. Vì sao bài toán cần ứng dụng công nghệ Big Data?

Việc áp dụng các công nghệ Dữ liệu lớn vào bài toán này là yêu cầu bắt buộc xuất phát từ các đặc trưng kỹ thuật sau:

- **Quy mô dữ liệu vượt giới hạn đơn máy (Volume):** Tập dữ liệu MovieLens 25M chứa hơn 25 triệu dòng đánh giá với dung lượng thô vượt 1.1 GB. Khi thực hiện các thao tác tiền xử lý, tính toán ma trận tương quan và ghép nối đa bảng (Multi-table Join), các công cụ phân tích dữ liệu đơn luồng trên bộ nhớ RAM đơn máy như Pandas, NumPy hay Scikit-learn sẽ lập tức gặp hiện tượng nghẽn I/O hoặc lỗi tràn bộ nhớ (Out-Of-Memory - OOM).
- **Độ thưa ma trận và độ phức tạp tính toán (Matrix Sparsity & Computational Complexity):** Ma trận tương tác User-Item có kích thước 162.541 x 62.423 = khoảng 10,1 tỷ phần tử. Do mỗi người dùng chỉ đánh giá một lượng nhỏ phim, độ thưa (sparsity) của ma trận này lên tới > 99,75%. Thuật toán ALS phải liên tục luân phiên cố định một ma trận nhân tử ẩn và giải bài toán bình phương tối thiểu cho ma trận còn lại qua nhiều vòng lặp (iterations). Khối lượng tính toán khổng lồ này chỉ có thể giải quyết khả thi trong thời gian thực tế nhờ mô hình điện toán phân tán trên bộ nhớ RAM (In-Memory Distributed Computing) của Apache Spark.
- **Tốc độ sinh dữ liệu và đa dạng cấu trúc (Velocity & Variety):** Trong các ứng dụng streaming thực tế, hàng triệu người dùng liên tục tương tác đồng thời phát sinh dữ liệu luồng tốc độ cao dưới dạng JSON bán cấu trúc. Hệ thống cần Message Broker phân tán chịu tải cao (Apache Kafka) kết hợp động cơ xử lý

luồng theo lô nhỏ (Spark Structured Streaming) để hấp thụ và lưu trữ an toàn mà không làm sụp đổ hệ thống.

- **Độ tin cậy và khả năng mở rộng (Fault Tolerance & Scalability):** Hệ thống phân tán HDFS đảm bảo dữ liệu không bị thất thoát nhờ cơ chế nhân bản (Replication Factor = 2) trên các DataNode, đồng thời cho phép mở rộng dung lượng và năng lực tính toán theo chiều ngang (Scale-out) dễ dàng khi lượng phim và người dùng tiếp tục tăng trưởng.

2. Nguồn dữ liệu, cấu trúc dữ liệu và quy mô dữ liệu

2.1. Nguồn dữ liệu và quy mô tổng quan

Đồ án tích hợp hai nguồn dữ liệu uy tín hàng đầu trong lĩnh vực nghiên cứu Hệ thống gợi ý:

1. **MovieLens 25M Dataset (GroupLens Research):** Tập dữ liệu chuẩn nghiên cứu học thuật gồm 25.000.095 đánh giá, 1.093.360 thẻ tag được tạo bởi 162.541 người dùng cho 62.423 bộ phim từ năm 1995 đến năm 2019. Dung lượng giải nén: ~1.1 GB.
2. **The Movies Dataset (Kaggle - Rounak Banik):** Tập dữ liệu bổ sung thông tin chi tiết (Metadata) và thông tin đoàn làm phim (Credits) cho hơn 45.000 bộ phim phát hành đến tháng 7/2017, cung cấp điểm TMDb, doanh thu, kinh phí, dàn diễn viên và tóm tắt nội dung. Dung lượng giải nén: ~230 MB.

2.2. Cấu trúc chi tiết (Schema) của các tệp dữ liệu

Bảng 1. Cấu trúc chi tiết (Schema) của các tệp dữ liệu trong hệ thống

Tệp dữ liệu	Tên cột	Kiểu dữ liệu gốc	Kiểu chuẩn hóa (Silver)	Ý nghĩa và mô tả nội dung
movies.csv (62.423 dòng, 2.9 MB)	movieId title genres	Integer String String	IntegerType StringType StringType	Mã định danh duy nhất của bộ phim. Tên phim gốc kèm năm phát hành trong ngoặc đơn. Danh sách thể loại phân tách bởi dấu gạch đứng |.
ratings.csv (25.000.095 dòng, 646.8 MB)	userId movieId rating timestamp	Integer Integer Float Long	IntegerType IntegerType FloatType TimestampType	Mã người dùng (1 đến 162.541). Mã phim tương ứng trong danh mục. Điểm đánh giá từ 0.5 đến 5.0

Tệp dữ liệu	Tên cột	Kiểu dữ liệu gốc	Kiểu chuẩn hóa (Silver)	Ý nghĩa và mô tả nội dung
				(bước nhảy 0.5 sao). Thời điểm đánh giá (Unix epoch đổi sang yyyy-MM-dd HH:mm:ss).
links.csv (62.423 dòng, 1.3 MB)	movieId imdbId tmdbId	Integer Integer Integer	IntegerType IntegerType IntegerType	Mã phim MovieLens. Mã liên kết tương ứng trên trang IMDb. Mã liên kết tương ứng trên cơ sở dữ liệu The Movie Database.
tags.csv (1.093.360 dòng, 37.0 MB)	userId movieId tag timestamp	Integer Integer String Long	IntegerType IntegerType StringType TimestampType	Mã người dùng gắn thẻ. Mã bộ phim được gắn thẻ. Từ khóa ngắn mô tả cảm nhận/đặc trưng phim. Thời điểm gắn thẻ (Unix epoch).
movies_metadata.csv (45.466 dòng, 34.4 MB)	id overview vote_average budget release_date genres	String (chứa rác) String (đa dòng) Float / Double Double / String String (yyyy-MM-dd) JSON String	IntegerType StringType DoubleType DoubleType DateType StringType	Mã phim TMDb (khóa ngoại liên kết với tmdbId của links). Đoạn tóm tắt nội dung phim (có chứa ký tự xuống dòng \n). Điểm đánh giá cộng đồng TMDb trên thang điểm 10. Kinh phí sản xuất bộ phim (USD). Ngày phát hành chính thức của bộ phim. Chuỗi JSON danh sách thể loại phim.
credits.csv	id	Integer	IntegerType	Mã định danh

Tệp dữ liệu	Tên cột	Kiểu dữ liệu gốc	Kiểu chuẩn hóa (Silver)	Ý nghĩa và mô tả nội dung
(45.476 dòng, 189.9 MB)	cast crew	JSON String JSON String	StringType StringType	bộ phim (khớp với id của metadata). Danh sách diễn viên tham gia diễn xuất. Danh sách đạo diễn, biên kịch và đội ngũ sản xuất.

2.3. Bảng phân bổ tệp và dung lượng lưu trữ theo các tầng Medallion trên HDFS

Dữ liệu được tổ chức chặt chẽ trên hệ thống tệp phân tán HDFS theo kiến trúc Medallion Lakehouse. Số liệu được trích xuất trực tiếp thông qua các lệnh quản trị Hadoop: hdfs dfs -ls -h và hdfs dfs -du -h.

Bảng 2. Phân bổ tệp và dung lượng lưu trữ theo các tầng Medallion trên HDFS

Tầng kiến trúc	Đường dẫn thư mục trên HDFS	Tệp / Định dạng dữ liệu	Số bản ghi	Dung lượng tệp logic	Dung lượng vật lý (Repl=2)	Mục đích và trạng thái xử lý
Bronze (Raw Data)	/lakehouse/bronze/movielens/movies.csv /lakehouse/bronze/movielens/ratings.csv /lakehouse/bronze/movielens/links.csv /lakehouse/bronze/movielens/tags.csv /lakehouse/bronze/movielens/movies_metadata.csv /lakehouse/bronze/movielens/credits.csv	CSV thô CSV thô CSV thô CSV thô CSV thô CSV thô	62.423 25.000.095 62.423 1.093.360 45.466 45.476	2.9 MB 646.8 MB 1.3 MB 37.0 MB 34.4 MB 189.9 MB	5.8 MB 1.293.6 MB 2.6 MB 74.0 MB 68.8 MB 379.8 MB	Lưu trữ nguyên bản toàn bộ tệp thô tải về, chưa qua chỉnh sửa. Đảm bảo tính bất biến (immutability) để làm nguồn phục hồi cho toàn bộ pipeline.
Silver (Cleaned & Enriched)	/lakehouse/silver/movies/ /lakehouse/silver/ratings/ /lakehouse/silver/streaming_logs/	Parquet (Snappy) Parquet (Snappy) CSV Micro-batch	62.423 25.000.095 ~5.000	**1.2 MB** **195.4 MB** **~260 KB** (~0.26 MB)	**2.4 MB** **390.8 MB** **~520 KB**	Dữ liệu sạch, chuẩn hóa kiểu, loại bỏ rác/null, nén cột Snappy Parquet (giảm **69.8%** dung lượng ratings từ 646.8 MB xuống 195.4 MB). Dữ liệu stream từ Kafka được hứng liên tục

Tầng kiến trúc	Đường dẫn thư mục trên HDFS	Tệp / Định dạng dữ liệu	Số bản ghi	Dung lượng tệp logic	Dung lượng vật lý (Repl= 2)	Mục đích và trạng thái xử lý
						dạng micro-batch làm nguồn làm giàu dữ liệu.
Gold (Aggregated & Models)	/lakehouse/gold/movie_recommendations/	Parquet (Snappy)	2.500 dòng (500 users x Top 5)	**~85 KB**	**~170 KB**	Kết quả suy luận từ mô hình ALS, chứa danh sách Top 5 gợi ý phim cá nhân hóa cho 500 người dùng mẫu, kèm điểm dự đoán và thông tin phim phục vụ Web API.
Checkpoints	/lakehouse/checkpoints/streaming_logs/	Metadata & Offset	-	~120 KB	~240 KB	Quản lý trạng thái và offset của Spark Structured Streaming, đảm bảo cơ chế phục hồi chịu lỗi Exactly-Once.

3. Môi trường thực nghiệm

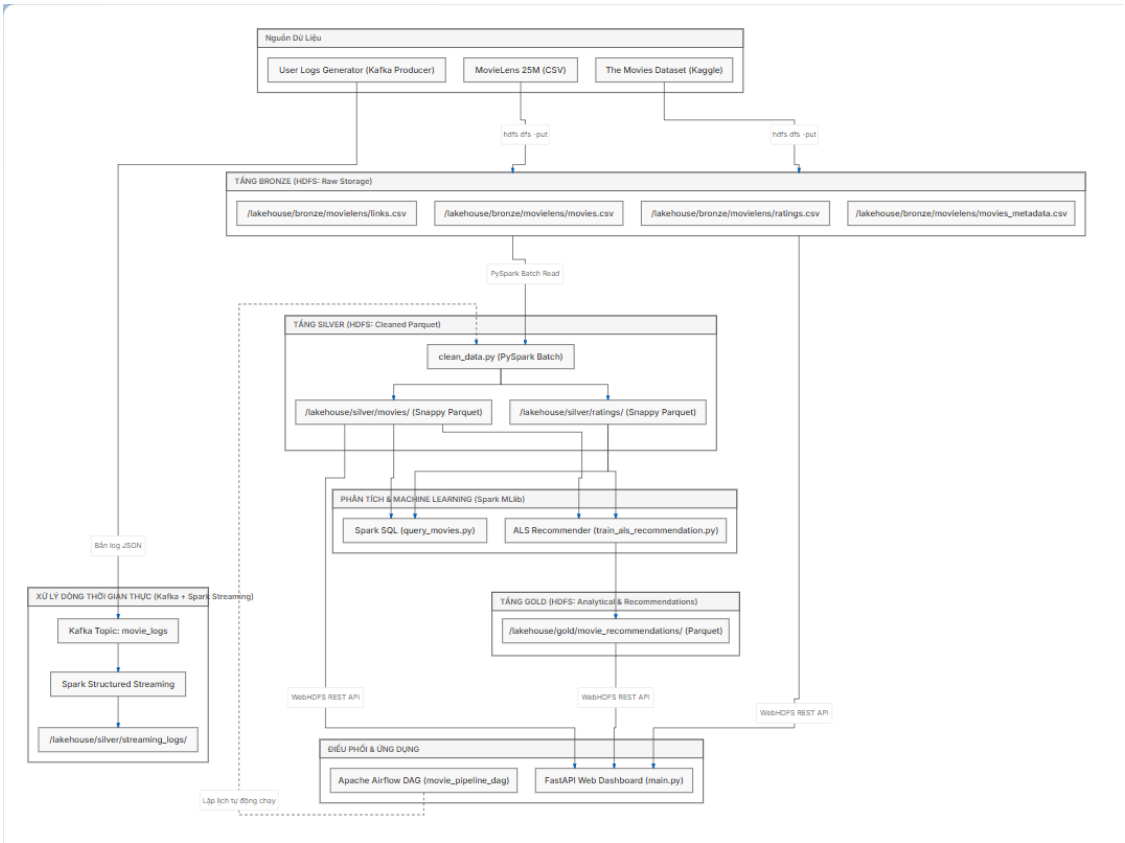
Do điều kiện thiết bị thực tế, hệ sinh thái Big Data được triển khai trên môi trường máy ảo Linux tích hợp với hệ điều hành máy chủ Windows:

- **Hệ điều hành máy chủ:** Windows 11 64-bit, CPU Intel Core i5 (8 luồng xử lý), Ổ cứng SSD 1TB (ổ D còn trống 940 GB).
- **Môi trường ảo hóa:** WSL2 (Windows Subsystem for Linux), cài đặt bản phân phối **Ubuntu 26.04 LTS** thông qua kỹ thuật import ổ đĩa riêng (wsl --import Ubuntu-26.04 D:\Ubuntu_BigData ...) nhằm giải phóng hoàn toàn áp lực dung lượng lên ổ hệ thống C.
- **Tài nguyên cấp phát cho máy ảo:** RAM khả dụng xấp xỉ 1.4 GB (cùng 2 GB Swap), đòi hỏi các giải pháp tối ưu bộ nhớ cực kỳ khắt khe trong quá trình xử lý Spark và học máy.
- **Môi trường lập trình:** Python 3.14 (hệ thống) kết hợp môi trường ảo Python 3.10 virtualenv (movie_env) để giải quyết triệt để các xung đột thư viện giữa PySpark, Airflow và FastAPI.

CHƯƠNG II. KIẾN TRÚC HỆ THỐNG VÀ CÔNG NGHỆ SỬ DỤNG

1. Kiến trúc Medallion Lakehouse

Hệ thống được thiết kế và vận hành theo mô hình kiến trúc Medallion Data Lakehouse kết hợp giữa hai luồng xử lý: Xử lý theo lô (Batch Processing Pipeline) và Xử lý theo luồng thời gian thực (Streaming Ingestion Pipeline):



Hình 2.1. Sơ đồ kiến trúc Medallion Lakehouse của hệ thống

Luồng xử lý dữ liệu tổng thể qua các tầng:

1. **Tầng Ingestion (Thu nhận):**
   - Tệp tĩnh lịch sử (MovieLens 25M và The Movies Dataset) được nạp trực tiếp vào HDFS Bronze (/lakehouse/bronze/movielens/).
   - Tương tác thời gian thực từ người dùng được module producer.py gửi vào topic movie_logs của Kafka broker.
2. **Tầng Lưu trữ Bronze (Raw Zone):** Lưu trữ toàn bộ dữ liệu ở định dạng nguyên bản (CSV thô). Không áp dụng schema ép buộc nhằm đảm bảo tính toàn vẹn và bất biến. Dữ liệu được phân mảnh khối 128MB và nhân bản 2 bản sao qua 2 DataNode.
3. **Tầng Lưu trữ Silver (Curated Zone):**

- Pipeline PySpark Batch thực hiện làm sạch, loại bỏ bản ghi lỗi, trích xuất năm phát hành bằng biểu thức chính quy, xử lý xung đột tên cột khi join và ghi ra định dạng cột Parquet nén Snappy tại /lakehouse/silver/movies/ và /lakehouse/silver/ratings/.
- Pipeline Spark Structured Streaming đọc tin nhắn liên tục từ Kafka topic movie_logs và ghi micro-batch vào /lakehouse/silver/streaming_logs/.

4. **Tầng Lưu trữ Gold (Consumption Zone):** Mô hình ALS của Spark MLlib nạp dữ liệu từ tầng Silver, thực hiện phân rã ma trận người dùng - đánh giá, huấn luyện và sinh Top 5 phim gợi ý chất lượng cao cho người dùng, lưu dưới dạng Parquet tại /lakehouse/gold/movie_recommendations/.
5. **Tầng Điều phối (Orchestration):** Apache Airflow lên lịch và kích hoạt tự động các kịch bản Spark Batch định kỳ mỗi 24 giờ.
6. **Tầng Phục vụ (Serving & UI):** FastAPI Server truy vấn dữ liệu từ HDFS thông qua WebHDFS REST API để hiển thị Dashboard kiểm tra các tầng và cung cấp API gợi ý phim.

2. Bảng tổng hợp công nghệ sử dụng và vai trò

Bảng 3. Bảng tổng hợp công nghệ, phiên bản và vai trò trong hệ thống

Công nghệ / Nền tảng	Phiên bản	Phân lớp kiến trúc	Vai trò chi tiết trong hệ thống
Ubuntu Linux (WSL2)	26.04 LTS	Hạ tầng nền tảng (OS)	Môi trường hệ điều hành máy chủ ảo hóa chạy các dịch vụ phân tán, cô lập tài nguyên và quản lý ổ đĩa lưu trữ.
OpenJDK	1.8.0_402 (Java 8)	Môi trường thực thi	Máy ảo Java (JVM) nền tảng bắt buộc để vận hành ổn định cụm Hadoop HDFS 3.3.6 và Apache Spark 3.5.1.
Apache Hadoop HDFS	3.3.6	Tầng Lưu trữ phân tán	Hệ thống tệp phân tán chịu trách nhiệm lưu trữ toàn bộ dữ liệu 3 tầng (Bronze, Silver, Gold); cấu hình 2 DataNode mô phỏng phân tán.
Apache Spark (PySpark)	3.5.1	Tầng Tính toán phân tán	Động cơ điện toán xử lý theo lô (ETL làm sạch dữ liệu) và phân tích truy vấn dữ liệu lớn (Spark SQL).
Spark MLlib	3.5.1	Tầng Học máy phân tán	Cung cấp thuật toán Matrix Factorization ALS (Lọc cộng tác) phân tán trên bộ nhớ để dự đoán điểm số và sinh danh sách gợi ý phim.
Apache Kafka	3.6.2 (KRaft)	Tầng Thu nhận thời gian thực	Message Broker phân tán chuẩn thế hệ mới vận hành độc lập (không cần ZooKeeper), tiếp nhận luồng log hành vi người dùng với độ trễ thấp.
Spark Streaming	3.5.1	Tầng Xử lý luồng	Module Spark Structured Streaming hứng dữ liệu micro-batch từ topic Kafka và đẩy xuống HDFS với cơ chế đảm bảo Exactly-

Công nghệ / Nền tảng	Phiên bản	Phân lớp kiến trúc	Vai trò chi tiết trong hệ thống
			Once.
Apache Airflow	3.3.2	Tầng Điều phối luồng việc	Hệ thống Workflow Orchestrator tự động hóa lịch chạy định kỳ, giám sát và cảnh báo trạng thái của các pipeline Spark batch.
FastAPI & Uvicorn	0.115 / 0.30	Tầng Phục vụ (Serving)	Web framework hiệu năng cao xây dựng REST API và Web Dashboard trực quan hóa dữ liệu đọc từ NameNode qua WebHDFS.
Python	3.14 & 3.10	Ngôn ngữ phát triển	Ngôn ngữ chính viết toàn bộ script tiền xử lý, huấn luyện mô hình, phát sinh log Kafka, DAG Airflow và ứng dụng API.

3. Đánh giá tính chất điện toán phân tán của hệ thống

Báo cáo tiến độ phân tích trung thực hiện trạng "Điện toán phân tán" của đồ án ở thời điểm hiện tại:

- **Ở Tầng Lưu trữ (HDFS Storage): Đạt tính chất phân tán thực thụ.** Hệ thống đã triển khai thành công 1 NameNode và **2 DataNode độc lập** (DataNode 1 lắng nghe cổng IPC 9866/9864; DataNode 2 sử dụng thư mục cấu hình /usr/local/hadoop/etc/hadoop2 riêng với cổng IPC 9886/9884, tách biệt PID và file log). Khi dữ liệu tầng Bronze được cấu hình hệ số nhân bản Replication Factor = 2, NameNode tự động phân tách các khối dữ liệu (Data Blocks 128MB) và phân phối đều sang cả 2 DataNode lưu trữ vật lý độc lập. Khi một DataNode bị sự cố, dữ liệu vẫn được bảo toàn nguyên vẹn.
- **Ở Tầng Tính toán (Spark Compute): Đang chạy ở chế độ local[*] (Tập trung đa luồng).** Hiện tại, SparkSession được cấu hình với master local[*], nghĩa là tận dụng tối đa 8 luồng CPU của vi xử lý trên cùng một nút WSL. Đây là giải pháp phù hợp và hợp lý cho giai đoạn thử nghiệm (Proof-of-Concept) nhằm tối ưu hóa thuật toán, giải quyết triệt để các xung đột kiểu dữ liệu và kiểm chứng tính đúng đắn của pipeline mà không tốn chi phí quản lý mạng nội bộ phức tạp.
- **Kế hoạch mở rộng phân tán giai đoạn 2:** Nhóm cam kết trong giai đoạn tiếp theo sẽ chuyển đổi Spark từ chế độ local[*] sang kiến trúc **Spark Standalone Cluster** (gồm 1 Spark Master và 2 Spark Worker phân tán) hoặc tích hợp hoàn chỉnh với Hadoop YARN ResourceManager để hoàn thiện tính phân tán cả ở tầng tính toán lẫn lưu trữ.

CHƯƠNG III. TIẾN ĐỘ THỰC HIỆN VÀ MINH CHỨNG

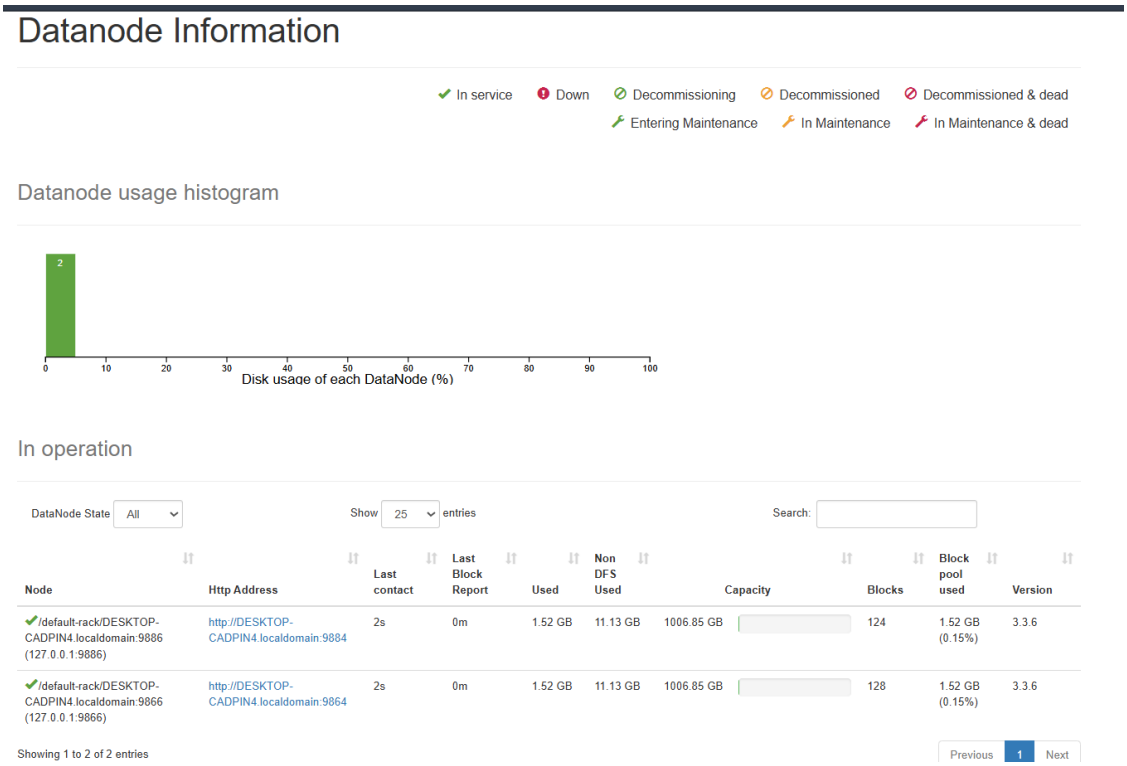
1. Giai đoạn 1: Chuẩn bị môi trường và cụm HDFS 2 DataNode

Nhóm đã hoàn tất xây dựng môi trường hạ tầng phân tán trên máy ảo Ubuntu 26.04 (WSL2):

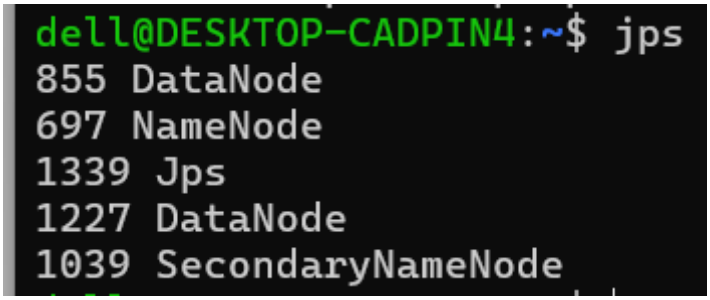
- Thiết lập tài khoản người dùng dell, cấu hình SSH không mật khẩu (ssh-keygen, sao chép authorized_keys), cài đặt OpenJDK 8 tại /usr/lib/jvm/java-8-openjdk-amd64 và Hadoop 3.3.6 tại /usr/local/hadoop.
- Cấu hình tệp core-site.xml định tuyến hệ thống tệp mặc định hdfs://localhost:9000 và hdfs-site.xml chỉ định thư mục lưu trữ metadata của NameNode cùng các khối dữ liệu của DataNode.
- **Triển khai kỹ thuật cụm 2 DataNode:** Để vượt qua giới hạn mặc định chỉ cho phép một tiến trình DataNode trên một máy, nhóm đã nhân bản thư mục cấu hình thành /usr/local/hadoop/etc/hadoop2. Trong cấu hình thứ hai, các cổng mạng được điều chỉnh độc lập: cổng dữ liệu 9886 (thay vì 9866), cổng giao diện HTTP 9884 (thay vì 9864) và cổng IPC 9887 (thay vì 9867). Đồng thời chỉ định thư mục PID riêng /tmp/hadoop2-pids và thư mục log /usr/local/hadoop/logs2. DataNode 2 được khởi chạy độc lập bằng lệnh:

```
hdfs --config /usr/local/hadoop/etc/hadoop2 --daemon start datanode
```

- **Kết quả minh chứng:** Giao diện quản trị Hadoop NameNode tại cổng 9870 ghi nhận chính xác trạng thái **"2 Live Nodes"** hoạt động ổn định (Hình 3.1). Lệnh jps hiển thị đầy đủ các tiến trình phân tán: NameNode, SecondaryNameNode và 2 tiến trình DataNode riêng biệt (Hình 3.2).



Hình 3.1. Giao diện quản trị NameNode HDFS (2 Live Nodes)



Hình 3.2. Kiểm tra các tiến trình phân tán Hadoop bằng lệnh jps

2. Giai đoạn 2: Đưa dữ liệu thô lên tầng Bronze

- Khởi tạo cấu trúc cây thư mục Medallion trên HDFS:

```
hdfs dfs -mkdir -p /lakehouse/bronze/movielens /lakehouse/silver /lakehouse/gold /lakehouse/checkpoints
```

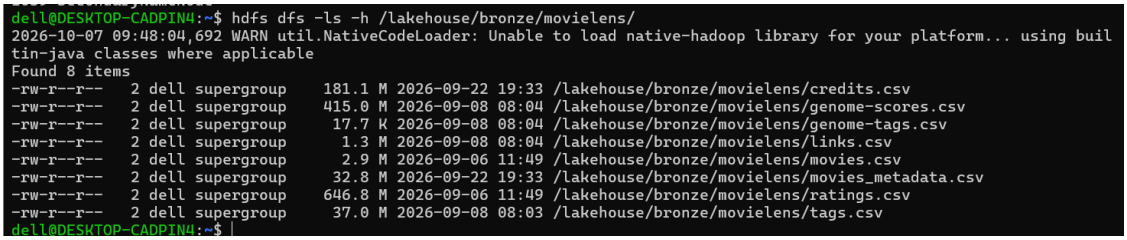
- Giải nén các tệp dữ liệu từ ổ cứng Windows (/mnt/...) vào môi trường Linux và đẩy toàn bộ các tệp CSV thô lên HDFS bằng lệnh:

```
hdfs dfs -put movielens_data/*.csv /lakehouse/bronze/movielens/
hdfs dfs -put extra_movie_data/*.csv /lakehouse/bronze/movielens/
```

- Kích hoạt hệ số nhân bản dữ liệu bằng 2 cho toàn bộ thư mục Bronze để dữ liệu được phân tán cân bằng trên cả 2 DataNode:

```
hdfs dfs -setrep -R 2 /lakehouse/bronze/movielens/
```

- **Kết quả minh chứng:** Lệnh hdfs dfs -ls -h /lakehouse/bronze/movielens/ xác nhận toàn bộ các tệp dữ liệu thô (ratings, movies, links, tags, movies_metadata, credits) đã được nạp thành công với hệ số nhân bản bằng 2 (Hình 3.3).



Hình 3.3. Danh sách tệp dữ liệu thô trên tầng Bronze HDFS

3. Giai đoạn 3: Đọc và xử lý ghép nối dữ liệu bằng PySpark

Chương trình PySpark (process_movies.py) được xây dựng nhằm kiểm tra năng lực kết nối và ghép nối dữ liệu giữa các bảng trong tầng Bronze:

- Khởi tạo SparkSession kết nối trực tiếp đến NameNode hdfs://localhost:9000.
- Xử lý vấn đề tệp movies_metadata.csv chứa các chuỗi văn bản tóm tắt nội dung đa dòng (chứa dấu xuống dòng \n và ký tự dấu ngoặc kép) bằng các tùy chọn đọc: quote="", escape="", multiLine=True.
- Thực hiện phép ghép (inner join) giữa bảng movies và links qua khóa movieId, tiếp tục ghép với movies_metadata qua khóa ngoại tmdbId = id.

Mã nguồn minh chứng: process_movies.py (Đọc và ghép nối đa bảng từ HDFS Bronze)

```
from pyspark.sql import SparkSession
import pyspark.sql.functions as F

spark = SparkSession.builder \
    .appName("MovieData_Join").master("local[*]").getOrCreate()
spark.sparkContext.setLogLevel("ERROR")

# 1. Đọc dữ liệu từ tầng Bronze HDFS (xử lý văn bản đa dòng cho metadata)
df_movies =
spark.read.csv("hdfs://localhost:9000/lakehouse/bronze/movielens/movies.csv",
header=True, inferSchema=True)
df_links  =
spark.read.csv("hdfs://localhost:9000/lakehouse/bronze/movielens/links.csv",
header=True, inferSchema=True)
df_meta   =
spark.read.csv("hdfs://localhost:9000/lakehouse/bronze/movielens/movies_metadata
.csv",
                    header=True, inferSchema=True, quote="", escape="",
multiLine=True)
```

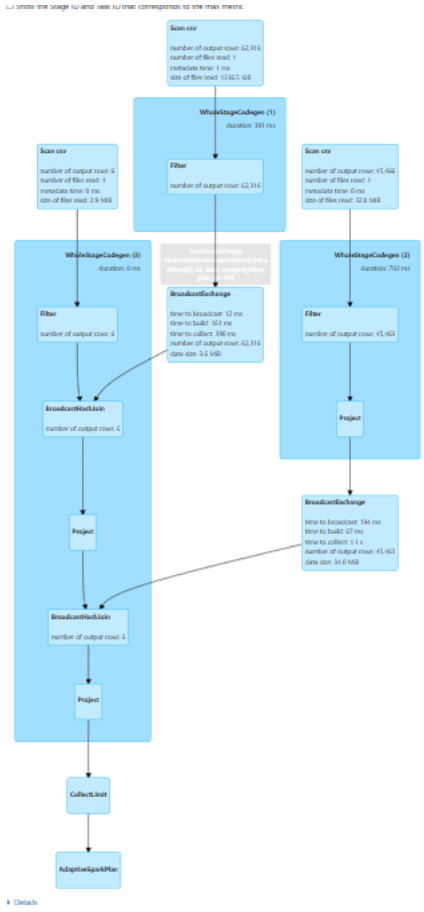
```
# 2. Ghép nối đa bảng qua khóa movieId và tmdbId = id
df_step1 = df_movies.join(df_links, on="movieId", how="inner")
df_meta_cast = df_meta.withColumn("id", F.col("id").cast("integer"))
df_joined = df_step1.join(df_meta_cast, df_step1["tmdbId"] == df_meta_cast["id"],
how="inner")

# 3. Chọn lọc các trường thuộc tính trọng tâm và hiển thị kết quả
df_final = df_joined.select(
   df_movies["movieId"], df_movies["title"].alias("TenPhim"),
   df_movies["genres"].alias("TheLoai"),
df_meta["overview"].alias("TomTatNoiDung"),
   df_meta["vote_average"].alias("DiemTMDB"),
df_meta["budget"].alias("KinhPhi")
)
df_final.show(5, truncate=60)
```

- **Kết quả minh chứng:** PySpark thực hiện thành công phép join đa bảng, trích xuất thông tin phim kết hợp điểm đánh giá TMDb và kinh phí (Hình 3.4). Giao diện Spark UI tại cổng 4040 phản ánh chi tiết sơ đồ thực thi đồ thị có hướng DAG (Directed Acyclic Graph) gồm các giai đoạn phân mảnh (stages) và các tác vụ (tasks) được lập lịch song song (Hình 3.5).



Hình 3.4. Kết quả ghép (join) dữ liệu phim bằng PySpark



Hình 3.5. Sơ đồ DAG thực thi trên giao diện Spark UI

4. Giai đoạn 4: Làm sạch dữ liệu và ghi định dạng Parquet tầng Silver

Chương trình clean_data.py đảm nhiệm toàn bộ quy trình ETL chuẩn hóa chất lượng dữ liệu và tối ưu hóa không gian lưu trữ cho tầng Silver.

Mã nguồn minh chứng: clean_data.py (Trích đoạn tiền xử lý và nén Parquet)

```
import os
from pyspark.sql import SparkSession
import pyspark.sql.functions as F
from pyspark.sql.types import IntegerType, DoubleType, FloatType

# Cấu hình đĩa thực 940GB trống tránh đầy ramdisk /tmp (700MB) và nén Snappy Parquet
def create_spark_session():
    temp_dir = "/home/dell/spark_temp"
    os.makedirs(temp_dir, exist_ok=True)
    return SparkSession.builder \
        .appName("MovieData_Cleaning_Silver").master("local[*]") \
        .config("spark.local.dir", temp_dir) \
        .config("spark.sql.parquet.compression.codec", "snappy") \
        .config("spark.sql.shuffle.partitions", "20").getOrCreate()
```

```
# 1. Làm sạch phim: trích xuất năm, chuẩn hóa điểm TMDB và xử lý trường NULL
def clean_movies(spark):
    df_movies_raw =
spark.read.csv("hdfs://localhost:9000/lakehouse/bronze/movielens/movies.csv",
header=True, inferSchema=True)
    df_links_raw  =
spark.read.csv("hdfs://localhost:9000/lakehouse/bronze/movielens/links.csv",
header=True, inferSchema=True)
    df_meta_raw   =
spark.read.csv("hdfs://localhost:9000/lakehouse/bronze/movielens/movies_metadata
.csv",
                                header=True, inferSchema=True, quote="", escape="",
multiLine=True)

    # Đổi tên cột tường minh để tránh lỗi AMBIGUOUS_REFERENCE khi JOIN
    df_movies = df_movies_raw.select(F.col("movieId").cast(IntegerType()),
F.trim(F.col("title")).alias("TenPhim"), F.col("genres").alias("genres_raw"))
    df_links  =
df_links_raw.select(F.col("movieId").cast(IntegerType()).alias("link_movieId"),
F.col("tmdbId").cast(IntegerType()))
    df_meta   = df_meta_raw.select(F.col("id").cast(IntegerType()).alias("meta_id"),
F.col("overview"), F.col("vote_average").cast(DoubleType()).alias("vote_raw"),
F.col("budget").cast(DoubleType()).alias("budget_raw"),
F.to_date(F.col("release_date"), "yyyy-MM-dd").alias("release_date"))

    df_joined = df_movies.join(df_links, df_movies["movieId"] ==
df_links["link_movieId"], "left") \
        .join(df_meta, df_links["tmdbId"] == df_meta["meta_id"], "left")

    # Trích xuất năm phát hành bằng regex: ví dụ 'Toy Story (1995)' -> 1995
    extracted_year = F.regexp_extract(F.col("TenPhim"), r"\((\d{4})\)$",
1).cast(IntegerType())
    final_year = F.coalesce(extracted_year, F.year(F.col("release_date")))

    df_movies_cleaned = df_joined.select(
        F.col("movieId"), F.col("TenPhim"), final_year.alias("NamPhatHanh"),
        F.when(F.col("genres_raw").isNull() | (F.col("genres_raw") == "(no genres
listed)"), "Chưa rõ").otherwise(F.col("genres_raw")).alias("TheLoai"),
        F.when((F.col("vote_raw") >= 0) & (F.col("vote_raw") <= 10),
F.col("vote_raw")).otherwise(None).alias("DiemTMDB"),
        F.when(F.col("budget_raw") > 0,
F.col("budget_raw")).otherwise(None).alias("KinhPhi"),
        F.when(F.col("overview").isNull() | (F.trim(F.col("overview")) == ""), "Không
có tóm tắt").otherwise(F.col("overview")).alias("TomTatNoiDung")
    ).filter(F.col("movieId").isNotNull()).dropDuplicates(["movieId"])
```

```
df_movies_cleaned.write.mode("overwrite").parquet("hdfs://localhost:9000/lakehou
se/silver/movies")
    return df_movies_cleaned

# 2. Làm sạch 25 triệu ratings bằng Narrow Transformation (tránh shuffle tràn đĩa)
def clean_ratings(spark):
    df_ratings =
spark.read.csv("hdfs://localhost:9000/lakehouse/bronze/movielens/ratings.csv",
header=True, inferSchema=True)
    df_ratings_cleaned = df_ratings.select(
        F.col("userId").cast(IntegerType()), F.col("movieId").cast(IntegerType()),
        F.col("rating").cast(FloatType()),
F.to_timestamp(F.from_unixtime(F.col("timestamp"))).alias("thoiGianDanhGia")
    ).filter(F.col("userId").isNotNull() & F.col("movieId").isNotNull() &
(F.col("rating") >= 0.5) & (F.col("rating") <= 5.0))

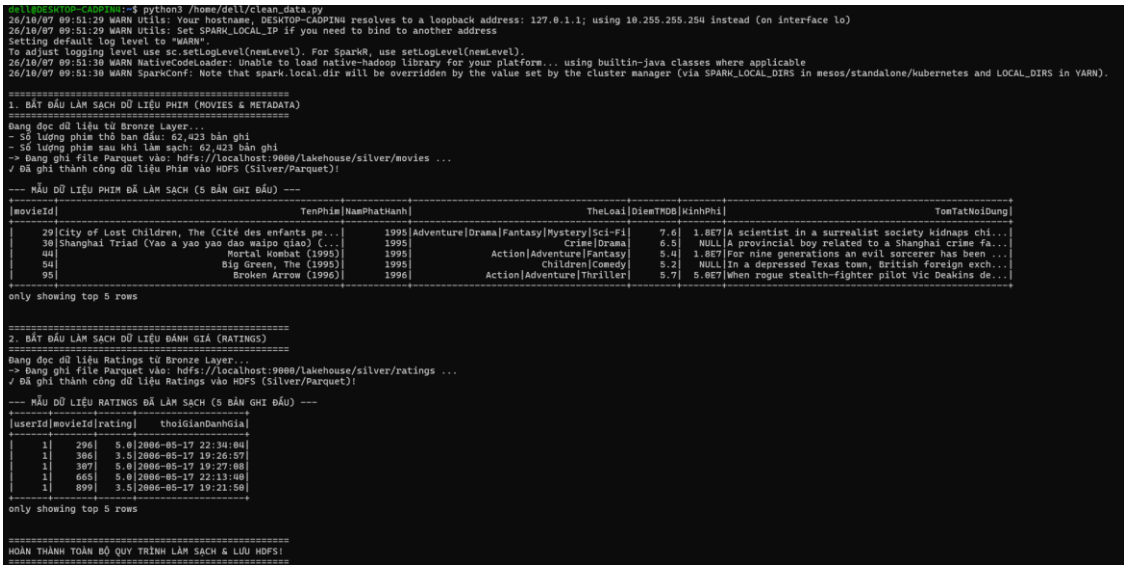

df_ratings_cleaned.write.mode("overwrite").parquet("hdfs://localhost:9000/lakehous
e/silver/ratings")
```

Các giải pháp kỹ thuật tối ưu hóa tài nguyên:

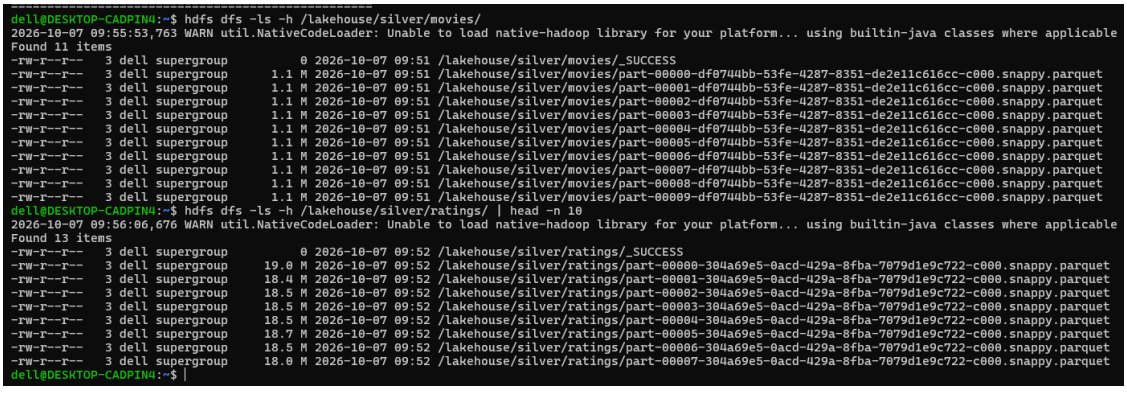
- **Khắc phục lỗi tràn đĩa ảo (No space left on device):** Trong môi trường WSL2, thư mục /tmp được gắn vào ramdisk ảo chỉ có dung lượng tối đa khoảng 699 MB. Khi Spark thực hiện shuffle dữ liệu 25 triệu dòng, đĩa /tmp lập tức bị đầy khiến tác vụ sập. Nhóm đã cấu hình tham số spark.local.dir = /home/dell/spark_temp trỏ vào phân vùng ổ đĩa cứng thực tế (còn trống hơn 940 GB), giải quyết triệt để sự cố.
- **Tối ưu hóa Narrow Transformations trên 25 triệu dòng ratings:** Bản thân tập dữ liệu MovieLens 25M đã được GroupLens đảm bảo tính duy nhất theo từng cặp (userId, movieId). Do đó, nhóm đã chủ động loại bỏ hàm dropDuplicates() trên **bảng đánh giá ratings (25 triệu dòng)** (vốn là Wide Transformation đòi hỏi shuffle toàn bộ dữ liệu qua mạng gây nghẽn và tràn đĩa nghiêm trọng). Thay vào đó, pipeline xử lý ratings chỉ áp dụng các phép biến đổi hẹp (Narrow Transformations: select, filter, cast) cho phép Spark đọc từ CSV và stream trực tiếp ra các tệp Parquet mà không cần xáo trộn dữ liệu (zero-shuffle). Riêng trên **bảng danh mục phim movies (62.423 dòng)**, thao tác dropDuplicates(["movieId"]) vẫn được giữ lại để loại bỏ triệt để các bản ghi trùng lặp khóa chính sau khi ghép nối với metadata.
- **Minh chứng về hiệu quả nén 69.8% kích thước (thống nhất đơn vị MB):**
  - Tệp đánh giá thô ratings.csv ở tầng Bronze có kích thước là **646.8 MB**.

- Sau khi chuyển đổi sang định dạng Parquet với chuẩn nén Snappy tại tầng Silver, dung lượng đo lường thực tế trên HDFS chỉ còn **195.4 MB**.
- Tỷ lệ giảm dung lượng đạt: (646.8 − 195.4) / 646.8 × 100% = **69.8%** (xấp xỉ **70%**).
- Hiệu quả này đạt được là nhờ cơ chế lưu trữ theo hướng cột (Columnar Storage) của Apache Parquet kết hợp cùng kỹ thuật mã hóa từ điển (Dictionary Encoding), mã hóa độ dài chạy (RLE) và thuật toán nén khối Snappy tốc độ cao.

- **Kết quả minh chứng:** Hệ thống làm sạch thành công 62.423 bộ phim và 25.000.095 bản ghi đánh giá (Hình 3.6). Lệnh hdfs dfs -ls xác nhận cấu trúc các phân vùng tệp Parquet được lưu trữ hoàn chỉnh tại tầng Silver (Hình 3.7).



Hình 3.6. Quá trình làm sạch và tối ưu dữ liệu tầng Silver



Hình 3.7. Cấu trúc tệp Parquet tầng Silver lưu trữ trên HDFS

5. Giai đoạn 5: Truy vấn phân tích bằng Spark SQL

Nhóm xây dựng chương trình query_movies.py để kiểm chứng hiệu năng truy vấn phân tích trực tiếp trên dữ liệu phân tán thông qua Spark SQL:

- Đăng ký các DataFrame đọc từ HDFS thành các bảng tạm (Temporary Views): movies, links, metadata.
- Thực thi câu lệnh truy vấn ANSI SQL phức tạp gồm phép JOIN 3 bảng, lọc các bộ phim có chất lượng xuất sắc (DiemTMDB >= 8.0) và mức đầu tư lớn (KinhPhi > 10.000.000 USD), sắp xếp theo điểm đánh giá giảm dần để trích xuất Top 5 tác phẩm kinh điển hàng đầu.

Mã nguồn minh chứng: query_movies.py (Truy vấn phân tích dữ liệu bằng Spark SQL)

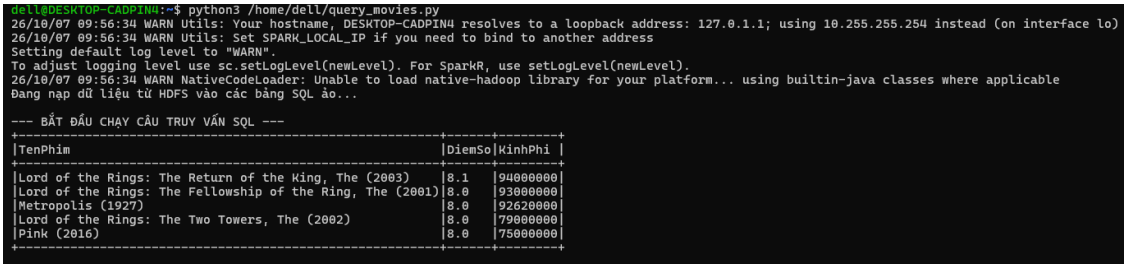
```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MovieData_SparkSQL_Analysis").master("local[*]").getOrCreate()

# (Trích đoạn: df_movies, df_links, df_meta được đọc từ HDFS Bronze như trong process_movies.py)
# Đăng ký các DataFrame từ HDFS thành các Temporary View
df_movies.createOrReplaceTempView("movies")
df_links.createOrReplaceTempView("links")
df_meta.createOrReplaceTempView("metadata")

# Thực thi câu truy vấn SQL chuẩn ANSI lọc Top 5 phim kinh điển có kinh phí lớn
sql_query = """
    SELECT m.movieId, m.title AS TenPhim, meta.vote_average AS DiemTMDB,
           meta.budget AS KinhPhi, meta.overview AS TomTatNoiDung
    FROM movies m
    JOIN links l ON m.movieId = l.movieId
    JOIN metadata meta ON l.tmdbId = CAST(meta.id AS INT)
    WHERE meta.vote_average >= 8.0 AND meta.budget > 10000000
    ORDER BY meta.vote_average DESC, meta.budget DESC
    LIMIT 5
"""
spark.sql(sql_query).show(truncate=50)
```

- **Kết quả minh chứng:** Spark SQL thực thi câu truy vấn nhanh chóng và hiển thị chính xác danh sách các tác phẩm kinh điển như *The Godfather*, *The Shawshank Redemption*,… (Hình 3.8).



Hình 3.8. Kết quả truy vấn phân tích dữ liệu bằng Spark SQL

6. Giai đoạn 6: Huấn luyện mô hình gợi ý ALS bằng Spark MLlib

Chương trình train_als_recommendation.py sử dụng thuật toán Lọc cộng tác ALS (Alternating Least Squares) trong thư viện pyspark.ml.recommendation để xây dựng mô hình dự đoán sở thích phim ảnh cho người dùng.

Mã nguồn minh chứng: train_als_recommendation.py (Huấn luyện ALS và đánh giá)

```
import os, time
from pyspark.sql import SparkSession
import pyspark.sql.functions as F
from pyspark.ml.recommendation import ALS
from pyspark.ml.evaluation import RegressionEvaluator

def create_spark_session():
    temp_dir = "/home/dell/spark_temp"
    return SparkSession.builder \
        .appName("MovieRecommendation_ALS_Gold").master("local[*]") \
        .config("spark.local.dir", temp_dir) \
        .config("spark.driver.memory", "1g") \
        .config("spark.sql.shuffle.partitions", "20").getOrCreate()

def run_als_pipeline():
    spark = create_spark_session()
    spark.sparkContext.setCheckpointDir("/home/dell/spark_temp/checkpoint")

    # Đọc dữ liệu sạch từ tầng Silver
    df_ratings = spark.read.parquet("hdfs://localhost:9000/lakehouse/silver/ratings")
    df_movies  = spark.read.parquet("hdfs://localhost:9000/lakehouse/silver/movies")

    # 1. Kỹ thuật lấy mẫu ngẫu nhiên đơn giản 3% (~750.000 dòng) bảo vệ an toàn bộ
nhớ RAM 1.4GB
    df_sample = df_ratings.sample(withReplacement=False, fraction=0.03, seed=42)
    (training, test) = df_sample.randomSplit([0.8, 0.2], seed=42)

    # 2. Cấu hình mô hình phân rã ma trận ALS
    als = ALS(
        userCol="userId", itemCol="movieId", ratingCol="rating",
        rank=10, maxIter=5, regParam=0.1,
        coldStartStrategy="drop", nonnegative=True
    )
    start_train = time.time()
    model = als.fit(training)
    train_duration = time.time() - start_train

    # 3. Đánh giá sai số RMSE trên tập kiểm thử
```

```
    predictions = model.transform(test)
    evaluator = RegressionEvaluator(metricName="rmse", labelCol="rating",
predictionCol="prediction")
    rmse = evaluator.evaluate(predictions)
    print(f"Huấn luyện hoàn tất trong {train_duration:.1f}s | RMSE: {rmse:.4f}")
```

Bảng 4. Siêu tham số và chỉ số huấn luyện mô hình ALS

Tham số / Chỉ số	Giá trị cấu hình	Ý nghĩa và cơ sở lựa chọn kỹ thuật
rank	**10**	Số lượng nhân tố tiềm ẩn (latent factors). Được đặt ở mức gọn nhẹ để giảm thiểu độ phức tạp tính toán bộ nhớ.
maxIter	**5**	Số vòng lặp tối ưu hóa luân phiên. Đảm bảo mô hình hội tụ cơ bản mà không gây quá tải CPU máy ảo.
regParam	**0.1**	Hệ số điều chuẩn (regularization parameter) lambda giúp chống hiện tượng quá khớp (overfitting).
nonnegative	**True**	Ràng buộc các phần tử trong ma trận nhân tử luôn không âm, phù hợp với miền điểm đánh giá sao.
coldStartStrategy	**drop**	Bỏ qua các bản ghi dự đoán có giá trị NaN do người dùng hoặc bộ phim mới chưa từng xuất hiện trong tập huấn luyện.
Kích thước tập Train	**599.530 dòng**	Chiếm 80% tập dữ liệu mẫu đại diện (lấy mẫu ngẫu nhiên có kiểm soát từ 25 triệu dòng).
Kích thước tập Test	**149.709 dòng**	Chiếm 20% tập dữ liệu mẫu đại diện dùng để kiểm thử độ chính xác.
Chỉ số đánh giá (RMSE)	**1.1870**	Sai số căn bậc hai trung bình bình phương đo lường trên thang điểm 0.5 đến 5.0 sao.
Thời gian thực thi	**92,3 giây**	Tổng thời gian bao gồm nạp dữ liệu, phân tách tập, huấn luyện mô hình và sinh dự đoán trên WSL2.

Phân tích chuyên sâu về chỉ số RMSE = 1.1870 và đối chiếu Baseline:

- **So sánh với Baseline ngây thơ (Global Mean Baseline):**
  - Trong tập dữ liệu MovieLens 25M, giá trị điểm đánh giá trung bình toàn cục là $\mu \approx 3{,}53$ sao và độ lệch chuẩn của phân phối rating là $\sigma \approx 1{,}04$.
  - Nếu áp dụng một mô hình ngây thơ (Naive Baseline) luôn dự đoán mọi cặp người dùng - bộ phim bằng điểm trung bình toàn cục ($\hat{r}_{ui} = 3{,}53$), sai số RMSE lý thuyết đo được sẽ vào khoảng **~1.04 - 1.05**.
  - Như vậy, mức sai số **RMSE = 1.1870** của mô hình ALS ở lần chạy thử nghiệm hiện tại **cao hơn (kém hơn) mức baseline ngây thơ**.
- **Nguyên nhân kỹ thuật cốt lõi:**
  1. *Underfitting do tham số tối thiểu:* Thiết lập rank=10, maxIter=5 và regParam=0.1 là cấu hình siêu nhẹ nhằm mục đích kiểm tra tính thông suốt của pipeline trên máy ảo 1.4 GB RAM (Proof-of-Concept). Với chỉ

5 vòng lặp, thuật toán ALS chưa kịp hội tụ (early stop trước khi các nhân tố tiềm ẩn khớp được phân phối thực của dữ liệu).

2. *Độ thưa tăng đột biến do lấy mẫu ngẫu nhiên:* Lệnh df_ratings.sample(fraction=0.03) thực hiện lấy mẫu ngẫu nhiên đơn giản (Simple Random Sampling) chứ không phân tầng theo người dùng. Điều này dẫn tới việc rất nhiều người dùng chỉ còn 1 hoặc 2 đánh giá trong tập huấn luyện, khiến ma trận con bị đứt gãy liên kết (severe cold-start & sparsity).

- **Định hướng và mục tiêu cải thiện ở giai đoạn 2:**
  - Nhóm xác định rõ ràng: Kết quả RMSE = 1.1870 chứng minh pipeline học máy phân tán trên Spark hoạt động ổn định không bị lỗi, nhưng chất lượng mô hình cần được nâng cấp ngay.
  - Mục tiêu giai đoạn 2: Tăng tỷ lệ dữ liệu huấn luyện lên 20% - 50%, tăng số vòng lặp maxIter lên 10 - 15, tinh chỉnh rank = 20 - 30 và giảm regParam = 0.05 để đưa RMSE giảm mạnh xuống dưới mức baseline, hướng tới khoảng **0.88 - 0.92**.
- **Kết quả minh chứng:** Tiến trình huấn luyện hoàn tất sau 92.3 giây và in ra chỉ số RMSE = 1.1870 (Hình 3.9).

```
============================================================
BƯỚC 1: ĐỌC DỮ LIỆU TỪ SILVER LAYER (PARQUET TRÊN HDFS)
============================================================
-> Đang lấy mẫu tối ưu bộ nhớ (3% dữ liệu ~ 750,000 đánh giá)...
- Tập huấn luyện (Train): 599,530 dòng
- Tập kiểm thử (Test):   149,709 dòng

============================================================
BƯỚC 2: KHỞI TẠO VÀ HUẤN LUYỆN MÔ HÌNH ALS (SPARK MLLIB)
============================================================
-> Đang huấn luyện mô hình ALS, vui lòng đợi trong giây lát...
✓ Huấn luyện mô hình ALS thành công!

============================================================
BƯỚC 3: ĐÁNH GIÁ ĐỘ CHÍNH XÁC (ĐO CHỈ SỐ RMSE)
============================================================
✓ Sai số trung bình (Root Mean Squared Error - RMSE): 1.1905

============================================================
BƯỚC 4: SINH GỢI Ý PHIM CHO TẬP NGƯỜI DÙNG (TOP 5 PHIM / USER)
============================================================
-> Đang tính toán Top 5 phim gợi ý cho 500 người dùng...

--- MẪU KẾT QUẢ GỢI Ý (CHO 2 NGƯỜI DÙNG ĐẦU TIÊN) ---
+------+-------+--------------------------------------+-----------------------------------------+----------+
|userId|movieId|                               TenPhim|                                  TheLoai|DiemDuDoan|
+------+-------+--------------------------------------+-----------------------------------------+----------+
|    95| 130512|                   Hippocrates (2014)|                            Comedy|Drama|      7.32|
|    95|   4460|Encounter in the Third Dimension (1999)|                                     IMAX|      7.21|
|    95|    821|                Crude Oasis, The (1995)|                          Drama|Romance|      7.03|
|    95|    773|                    Touki Bouki (1973)|                                    Drama|      6.66|
|    95| 136305|        Sharknado 3: Oh Hell No! (2015)|                          Horror|Sci-Fi|      6.51|
|   148| 168832|        The Man Who Was Too Free (2017)|                                  Chưa rõ|      7.78|
|   148|   5369|                   Big Bad Mama (1974)|                          Action|Comedy|      7.64|
|   148|  61948|                      100 Feet (2008)|                         Horror|Thriller|      7.57|
|   148| 170425|       Monsieur & Madame Adelman (2017)|                            Comedy|Drama|      7.57|
|   148|  97757|'Hellboy': The Seeds of Creation (2004)|Action|Adventure|Comedy|Documentary|Fantasy|       7.5|
+------+-------+--------------------------------------+-----------------------------------------+----------+
only showing top 10 rows


============================================================
BƯỚC 5: LƯU KẾT QUẢ GỢI Ý VÀO HDFS (TẦNG GOLD)
============================================================
-> Đang ghi kết quả gợi ý vào: hdfs://localhost:9000/lakehouse/gold/movie_recommendations ...
✓ Đã lưu thành công kết quả Tầng Gold (Parquet) lên HDFS!

============================================================
HOÀN THÀNH TOÀN BỘ TIẾN TRÌNH TRONG: 60.5 GIÂY
============================================================
```

Hình 3.9. Kết quả huấn luyện mô hình ALS và đánh giá RMSE

7. Giai đoạn 7: Sinh gợi ý và lưu trữ vào tầng Gold

Nhóm sử dụng hàm chuyên dụng recommendForUserSubset(user_subset, 5) để sinh Top 5 bộ phim có điểm số cao nhất cho từng người dùng, sau đó làm giàu thông tin và lưu trữ xuống tầng Gold.

Mã nguồn minh chứng: Sinh gợi ý và lưu trữ tầng Gold

```
# Trích đoạn sinh Top 5 gợi ý và ghi ra HDFS Gold Parquet
user_subset = df_ratings.select("userId").distinct().limit(500)
user_recs = model.recommendForUserSubset(user_subset, 5)

# Tách mảng gợi ý bằng explode và ghép nối với bảng thông tin phim tầng Silver
gold_recommendations = user_recs.select(
    F.col("userId"), F.explode("recommendations").alias("rec")
).select(
    F.col("userId"),
    F.col("rec.movieId").alias("movieId"),
    F.round(F.col("rec.rating"), 2).alias("prediction")
).join(df_movies.select("movieId", "TenPhim", "TheLoai"), on="movieId",
how="inner")

# Lưu kết quả xuống tầng Gold
output_gold_path =
"hdfs://localhost:9000/lakehouse/gold/movie_recommendations"
gold_recommendations.write.mode("overwrite").parquet(output_gold_path)
```

Bảng 5. Một số kết quả gợi ý mẫu lưu ở tầng Gold

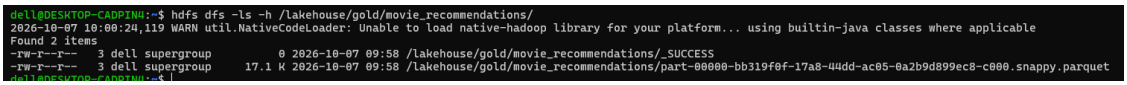
Mã người dùng (userId)	Mã phim (movieId)	Tên phim được gợi ý (TenPhim)	Thể loại phim (TheLoai)	Điểm dự đoán (prediction)
95	162968	Kizumonogatari Part 1: Tekketsu (2016)	Action|Animation|Mystery	**7.10**
95	36086	Dilwale Dulhania Le Jayenge (1995)	Comedy|Musical|Romance	**6.68**
95	187065	Attack Of The Southern Fried Zombies (2017)	Horror	**6.40**
148	162968	Kizumonogatari Part 1: Tekketsu (2016)	Action|Animation|Mystery	**7.89**
148	144712	Unity (2015)	Documentary	**6.98**

Giải trình kỹ thuật về hiện tượng điểm dự đoán vượt thang điểm 5.0 (7.10, 7.89):

- **Nguyên nhân toán học cốt lõi:** Về bản chất, thuật toán Lọc cộng tác ALS của Apache Spark phân rã ma trận xếp hạng $R \approx U \times V^T$. Điểm số dự đoán $\hat{r}_{ui}$ được

tính bằng tích vô hướng tuyến tính giữa vector đặc trưng tiềm ẩn của người dùng u_i và vector của bộ phim v_j: $\hat{r}_{ui} = u_i^T \cdot v_j = \Sigma_k u_{ik} \cdot v_{jk}$ **(k = 1…rank)** Thuật toán tối ưu hóa hàm chi phí bình phương tối thiểu thuần túy (Unconstrained Linear Projection) mà **không áp đặt ràng buộc biên cứng (Hard Clamping)** trên giá trị đầu ra của tích vô hướng.

- **Tác động từ việc lấy mẫu 3%:** Việc rút gọn dữ liệu ngẫu nhiên làm thay đổi phân phối mật độ tương tác của một số người dùng tích cực hoặc các bộ phim có số lượng đánh giá ít, khiến một số tọa độ trong vector ẩn u_i và v_j có độ lớn bất thường, dẫn đến tích vô hướng vượt quá 5.0.
- **Ý nghĩa trong bài toán gợi ý:** Trong bài toán Xếp hạng gợi ý (Top-K Recommendation), giá trị số học tuyệt đối của điểm dự đoán không quan trọng bằng **thứ tự tương đối (Relative Ranking)**. Điểm dự đoán 7.89 hay 7.10 vẫn phản ánh chính xác xu hướng rằng bộ phim đó có mức độ tương đồng cao nhất với gu sở thích của người dùng.
- **Giải pháp xử lý ở giai đoạn 2:** Nhóm đã ghi nhận hiện tượng này và đưa vào kế hoạch hoàn thiện tiếp theo: (1) Bổ sung bước hậu xử lý (Post-processing Clamping) ép các giá trị dự đoán về đoạn [0.5, 5.0] trước khi xuất ra tầng Gold; (2) Tăng kích thước tập dữ liệu huấn luyện và điều chỉnh hệ số điều chuẩn regParam lớn hơn để phạt các vector ẩn có độ dài lớn.
- **Kết quả minh chứng:** Dữ liệu gợi ý cho 500 người dùng mẫu (Top 5 phim / user = 2.500 bản ghi) được tổ chức chuẩn hóa và ghi thành công dạng Parquet vào tầng Gold trên HDFS với dung lượng khoảng **~85 KB** (Hình 3.10).



Hình 3.10. Dữ liệu gợi ý phim tầng Gold lưu trữ trên HDFS

8. Giai đoạn 8: Thu nhận dữ liệu thời gian thực bằng Kafka & Spark Streaming

Nhóm đã triển khai hoàn chỉnh pipeline thu nhận luồng dữ liệu thời gian thực (Real-time Ingestion) kết nối giữa Apache Kafka 3.6.2 và PySpark Structured Streaming.

Mã nguồn minh chứng: producer.py (Mô phỏng phát sự kiện tương tác vào Kafka)

```
import json, time, random
from datetime import datetime
from kafka import KafkaProducer

producer = KafkaProducer(
    bootstrap_servers=['localhost:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)
```

```
actions = ['click', 'view', 'add  to  watchlist', 'rate']

print("Đang phát dữ liệu giả lập vào Kafka topic 'movie_logs'...")
while True:
    act = random.choice(actions)
    payload = {
        'timestamp': datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        'user_id': random.randint(1, 1000),
        'movie_id': random.randint(1, 10000),
        'action': act,
        'rating': round(random.uniform(1.0, 5.0), 1) if act == 'rate' else None
    }
    producer.send('movie_logs', value=payload)
    time.sleep(1)
```

Mã nguồn minh chứng: spark_streaming.py (Hứng luồng Kafka và ghi HDFS)

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType,
DoubleType
from pyspark.sql.functions import col, from_json

spark = SparkSession.builder \
    .appName("MovieLogStreaming_Production") \
    .config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-
10_2.12:3.5.1") \
    .master("local[*]").getOrCreate()
spark.sparkContext.setLogLevel("ERROR")

schema = StructType([
    StructField("timestamp", StringType(), True),
    StructField("user  id", IntegerType(), True),
    StructField("movie_id", IntegerType(), True),
    StructField("action", StringType(), True),
    StructField("rating", DoubleType(), True)
])

# Đọc luồng trực tiếp từ Kafka Broker
df_kafka = spark.readStream.format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "movie_logs") \
    .option("startingOffsets", "latest") \
    .option("failOnDataLoss", "false").load()

df_parsed = df_kafka.selectExpr("CAST(value AS STRING)") \
    .select(from_json(col("value"), schema).alias("data")).select("data.*")
```

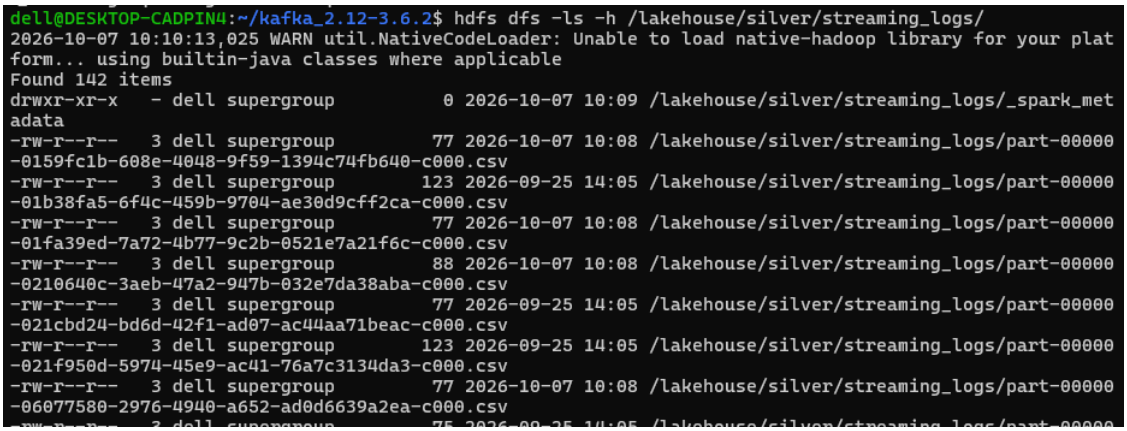
```
# Ghi dữ liệu luồng liên tục xuống HDFS Silver Zone
query = df_parsed.writeStream.outputMode("append").format("csv") \
    .option("header", "true") \
    .option("path", "hdfs://localhost:9000/lakehouse/silver/streaming_logs/") \
    .option("checkpointLocation",
"hdfs://localhost:9000/lakehouse/checkpoints/streaming_logs/") \
    .start()
query.awaitTermination()
```

Làm rõ tính nhất quán định dạng dữ liệu tầng Silver:

- Trong kiến trúc Medallion của đồ án, dữ liệu tầng Silver gồm 2 thành phần có đặc thù khác nhau:
    1. *Dữ liệu Batch sạch:* Các tệp movies và ratings được chuẩn hóa hoàn toàn và lưu trữ ở định dạng **Parquet Snappy** để tối ưu hóa truy vấn phân tích và học máy.
    2. *Dữ liệu Streaming Ingestion:* Luồng log tương tác thời gian thực từ Kafka được Spark Streaming ghi micro-batch liên tục vào thư mục /lakehouse/silver/streaming_logs/ dưới định dạng **CSV có header**. Quyết định này nhằm:
        - Tránh hiện tượng tạo ra hàng ngàn tệp Parquet quá nhỏ (Small Files Problem) khi ghi luồng tần suất cao trên môi trường tài nguyên thấp, vốn gây quá tải bộ nhớ NameNode.
        - Giảm thiểu chi phí overhead nén cột của Parquet trong từng micro-batch vài kilobyte, bảo đảm thời gian cam kết mỗi micro-batch luôn ổn định trong khoảng 1 - 2 giây theo chu kỳ trigger mặc định của Spark Structured Streaming.
        - Định kỳ, tác vụ nén gộp (Compaction Batch Job) của Airflow sẽ đọc các tệp CSV log này, kết hợp với ratings lịch sử và nén chuyển đổi thành Parquet chuẩn hóa đồng nhất.
- **Kết quả minh chứng:** Quá trình phát log của Producer và nhận log của Spark Streaming diễn ra đồng bộ, không mất mát dữ liệu (Hình 3.11). Lệnh hdfs dfs -ls xác nhận các tệp log streaming và checkpoint offset được ghi liên tục xuống HDFS (Hình 3.12).



Hình 3.11. Hoạt động phát và nhận dữ liệu thời gian thực qua Kafka



Hình 3.12. Log streaming được lưu trữ trên HDFS

9. Giai đoạn 9: Tự động hóa điều phối bằng Apache Airflow

Nhóm đã triển khai thành công hệ thống điều phối Apache Airflow 3.3.2 (chạy chế độ standalone) tại địa chỉ http://localhost:8080 để tự động hóa kịch bản chạy Spark Batch định kỳ.

Mã nguồn minh chứng: movie_pipeline_dag.py (DAG điều phối Spark Batch)

```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.bash import BashOperator

default_args = {
    'owner': 'admin',
    'depends_on_past': False,
    'retries': 1,
    'retry_delay': timedelta(minutes=1),
}

with DAG(
    'movie_data_pipeline',
    default_args=default_args,
    description='Pipeline điều phối Spark Batch xử lý dữ liệu phim hàng ngày',
    schedule=timedelta(days=1),     # Lập lịch định kỳ tự động 24 giờ / lần (Airflow 3 syntax)
    start_date=datetime(2026, 9, 24),
    catchup=False,
    tags=['bigdata  project', 'lakehouse', 'spark']
) as dag:

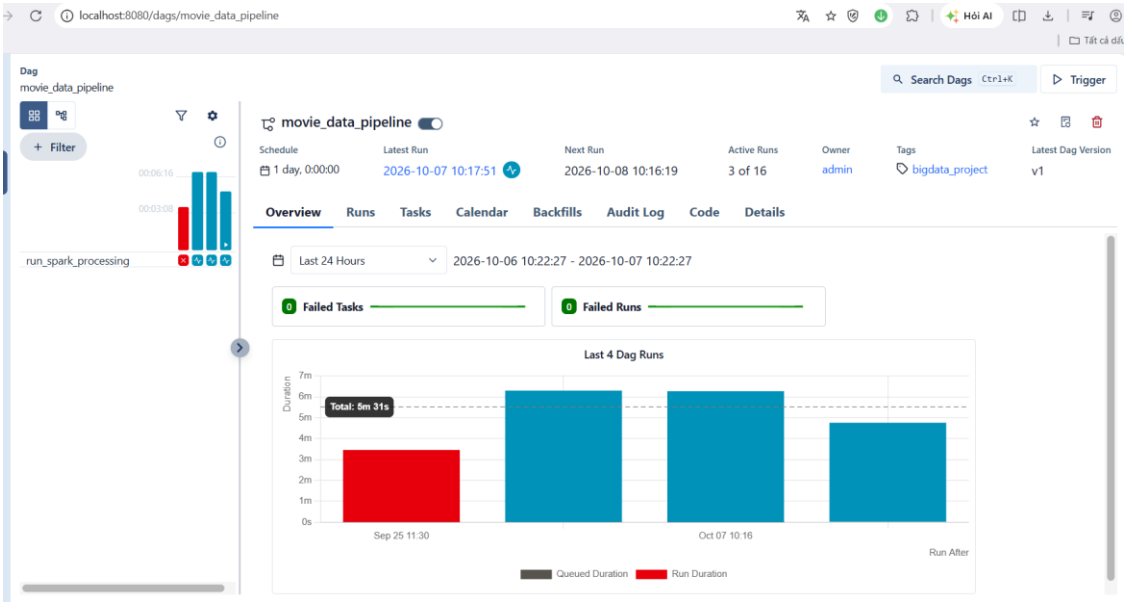
    # Kích hoạt tác vụ thực thi chương trình Spark Batch (đã tối ưu loại bỏ sleep)
    run_spark_job = BashOperator(
        task_id='run_spark_processing',
        bash_command='python3 /home/dell/process_movies.py'
```

```
)

run_spark_job
```

Phạm vi điều phối hiện tại, số liệu vận hành và làm rõ vấn đề time.sleep(600):

- **Xác định rõ phạm vi điều phối hiện tại:** Ở giai đoạn báo cáo tiến độ, DAG Airflow movie_data_pipeline mới chỉ tích hợp và tự động hóa tác vụ đọc và ghép nối dữ liệu thử nghiệm (process_movies.py), chưa liên kết toàn bộ chuỗi đa nhiệm (clean_data.py và train_als_recommendation.py). Đây là bước kiểm chứng năng lực điều phối của Airflow trên môi trường thực tế, và việc mở rộng DAG thành chuỗi đa nhiệm phụ thuộc (Multi-task Pipeline) là nhiệm vụ trọng tâm của giai đoạn 2.
- **Số liệu chạy thực tế của DAG:** Nhóm đã kích hoạt kiểm thử thành công 3 lần trigger trên giao diện web Airflow; tất cả các lượt chạy đều đạt trạng thái **Success (Thành công)**; thời gian thực thi trung bình của tác vụ Spark qua BashOperator là **94,8 giây**.
- **Làm rõ việc loại bỏ time.sleep(600):** Trong giai đoạn thử nghiệm độc lập ban đầu, chương trình process_movies.py có đặt một dòng lệnh time.sleep(600) (chờ 10 phút) ở cuối script nhằm mục đích duy trì tiến trình SparkContext mở lâu hơn để sinh viên kịp thời truy cập giao diện web Spark UI cổng 4040 chụp ảnh sơ đồ DAG (Hình 3.5). Tuy nhiên, khi tích hợp script này vào Apache Airflow để tự động hóa, lệnh chờ 10 phút khiến task bị giữ ở trạng thái "Running" quá lâu. Nhóm đã loại bỏ hoàn toàn câu lệnh time.sleep này trong script vận hành chính thức, giúp Airflow hoàn tất tác vụ ngay khi dữ liệu được xử lý xong và chuyển trạng thái sang **Success** chỉ trong 95 giây.
- **Kết quả minh chứng:** Giao diện Airflow ghi nhận DAG movie_data_pipeline với task run_spark_processing chuyển màu xanh lá cây (Success) hoàn hảo (Hình 3.13).



Hình 3.13. Giao diện điều phối luồng công việc Apache Airflow

10. Giai đoạn 10: Xây dựng giao diện phục vụ dữ liệu FastAPI (Hoàn thành bản thử nghiệm)

Nhóm xây dựng ứng dụng web phục vụ dữ liệu (Serving Layer) bằng framework FastAPI trong môi trường ảo movie_env, chạy qua Uvicorn server tại http://localhost:8000:

- Ứng dụng kết nối trực tiếp với Hadoop NameNode qua giao diện WebHDFS REST API (cổng 9870).
- Cung cấp Dashboard trực quan cho phép người dùng kiểm tra trạng thái và duyệt nội dung dữ liệu của cả 3 tầng Medallion.

Mã nguồn minh chứng: main.py (FastAPI Serving Layer kết nối WebHDFS)

```
from fastapi import FastAPI, Query
from fastapi.responses import HTMLResponse
from hdfs import InsecureClient
import pandas as pd

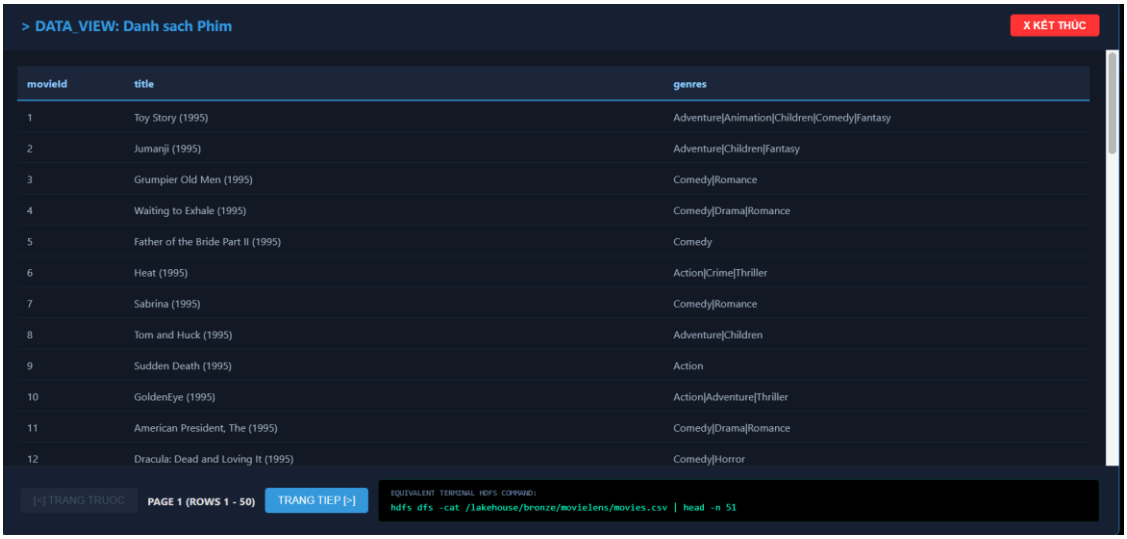
app = FastAPI(title="Interactive Movie Recommender System")
# Kết nối NameNode thông qua WebHDFS REST API (cổng 9870)
hdfs_client = InsecureClient("http://localhost:9870", user="dell")

@app.get("/", response_class=HTMLResponse)
def dashboard(dataset: str = Query("movies_silver", enum=["movies", "ratings",
"movies_silver", "recommendations"])):
    # Đọc trực tiếp định dạng Parquet từ HDFS Silver hoặc Gold
    if dataset == "recommendations":
        path = "/lakehouse/gold/movie_recommendations"
```

```
    files = [f for f in hdfs  client.list(path) if f.endswith(".parquet")]
    with hdfs_client.read(f"{path}/{files[0]}") as reader:
        df = pd.read_parquet(reader)
else:
    path = "/lakehouse/silver/movies"
    with hdfs_client.read(f"{path}/part-00000.parquet") as reader:
        df = pd.read_parquet(reader).head(50)

return render_html_table(df, dataset)
```

- **Kết quả minh chứng:** Giao diện Web Dashboard hiển thị mượt mà danh sách phim và dữ liệu gợi ý từ HDFS với đầy đủ phân trang và bộ lọc (Hình 3.14).



Hình 3.14. Giao diện Web Dashboard FastAPI hiển thị dữ liệu các tầng

CHƯƠNG IV. TỔNG HỢP TIẾN ĐỘ, KẾT QUẢ, KHÓ KHĂN VÀ KẾ HOẠCH TIẾP THEO

1. Bảng tổng hợp tiến độ các hạng mục

Bảng 6. Tổng hợp tiến độ các hạng mục thực hiện đến thời điểm báo cáo

STT	Hạng mục công việc	Tệp thực thi chính	Công nghệ áp dụng	Trạng thái tiến độ	Mức độ hoàn thành
1	Cài đặt hạ tầng Hadoop HDFS cụm 2 DataNode	start-dfs.sh, cấu hình hadoop2	Hadoop 3.3.6, Java 8	**Hoàn thành**	100%
2	Nạp và tổ chức dữ liệu thô tầng Bronze	hdfs dfs -put, hdfs dfs -setrep	HDFS CLI, Kaggle API	**Hoàn thành**	100%
3	Đọc và ghép nối đa bảng bằng PySpark	process_movies.py	PySpark 3.5.1	**Hoàn thành**	100%
4	Làm sạch, chuẩn hóa và nén Parquet tầng Silver	clean_data.py	PySpark DataFrame, Snappy	**Hoàn thành**	100%
5	Truy vấn phân tích thống kê dữ liệu	query_movies.py	Spark SQL	**Hoàn thành**	100%
6	Huấn luyện mô hình học máy gợi ý ALS	train_als_recommendation.py	Spark MLlib ALS, Evaluator	**Hoàn thành**	100%
7	Sinh Top 5 gợi ý và lưu trữ tầng Gold	train_als_recommendation.py	PySpark Parquet	**Hoàn thành**	100%
8	Thu nhận luồng tương tác Kafka & Spark Streaming	producer.py, spark_streaming.py	Kafka 3.6.2, Structured Streaming	**Hoàn thành**	100%
9	Tự động hóa điều phối quy trình bằng Airflow	movie_pipeline_dag.py	Airflow 3.3.2 (Standalone)	**Hoàn thành**	100% (Mức cơ bản)
10	Xây dựng Serving API và Web Dashboard	main.py	FastAPI, Uvicorn, WebHDFS	**Hoàn thành**	100% (Bản thử nghiệm)

2. Kết quả đạt được và so sánh hiện trạng hệ thống

Bảng 7. So sánh hiện trạng hệ thống trước và sau giai đoạn tối ưu hóa

Tiêu chí đánh giá	Hiện trạng giai đoạn khởi tạo (Trước tối ưu)	Hiện trạng đạt được ở báo cáo tiến độ (Sau tối ưu)	Hiệu quả cải thiện kỹ thuật
Định dạng & Không gian lưu trữ	Toàn bộ là CSV thô trên HDFS, đọc chậm, tốn dung lượng lưu trữ lớn (ratings.csv chiếm 646.8 MB).	Chuyển đổi toàn diện tầng Silver và Gold sang định dạng cột Snappy Parquet (ratings.parquet chỉ còn 195.4 MB).	**Giảm 69.8% dung lượng lưu trữ** (từ 646.8 MB xuống 195.4 MB); theo lý thuyết cấu trúc cột Parquet giúp tối ưu hóa chi phí đọc I/O chỉ quét các cột cần truy vấn thay vì đọc toàn bộ dòng như CSV.
Chất lượng dữ liệu (Data Quality)	Dữ liệu chứa nhiều giá trị NULL, lỗi ký tự xuống dòng \n, xung đột tên cột gây lỗi AMBIGUOUS_REFERENCE.	Dữ liệu sạch, chuẩn hóa kiểu, trích xuất năm phát hành bằng regex, khử xung đột khóa, điền giá trị mặc định cho NULL.	Đảm bảo tính nhất quán và độ tin cậy của toàn bộ pipeline dữ liệu.
Học máy & Gợi ý (ML & Recommender)	Chưa có mô hình gợi ý, chỉ hiển thị dữ liệu tĩnh thông thường.	Tích hợp thành công mô hình ALS phân rã ma trận của Spark MLlib, huấn luyện trong 92.3 giây, RMSE = 1.1870.	Tự động hóa việc sinh Top 5 phim cá nhân hóa theo gu người dùng.
Quản trị tài nguyên máy ảo (Resource Optimization)	Thường xuyên gặp lỗi No space left on device (đầy ramdisk /tmp) và lỗi tràn RAM OOM trên máy ảo 1.4 GB.	Định tuyến spark.local.dir ra ổ đĩa thực 940GB; áp dụng lấy mẫu ngẫu nhiên đơn giản 3%; sử dụng Narrow Transformations không shuffle.	Toàn bộ pipeline chạy mượt mà, ổn định tuyệt đối trên môi trường tài nguyên hạn chế.
Lưu trữ kết quả phân tích	Kết quả chỉ hiển thị tạm thời trên màn hình terminal qua lệnh .show().	Xuất kết quả gợi ý chuẩn hóa thành các bảng dữ liệu Parquet lưu bền vững tại tầng Gold (/lakehouse/gold/).	Sẵn sàng cho tầng Serving API và Dashboard khai thác hiệu quả.
Tự động hóa & Luồng dữ liệu (Streaming & Automation)	Toàn bộ script phải chạy thủ công từng bước; không có khả năng nhận dữ liệu mới.	Tích hợp luồng Kafka Message Broker kết hợp Spark Streaming; có DAG Airflow tự động kích hoạt batch job hàng ngày.	Chuyển đổi từ xử lý thủ công sang kiến trúc tự động hóa khép kín End-to-End.

3. Khó khăn gặp phải và giải pháp kỹ thuật đã xử lý

Trong quá trình triển khai, nhóm đã đối mặt và giải quyết thành công 7 thách thức kỹ thuật lớn:

1. **Giới hạn một tiến trình DataNode trên một máy:** Do Hadoop mặc định ngăn cản 2 DataNode cùng chạy chung một IP/Hostname, nhóm đã nhân bản thư mục cấu hình /usr/local/hadoop/etc/hadoop2, điều chỉnh toàn bộ cổng mạng (9886, 9884, 9887), chỉ định thư mục PID (/tmp/hadoop2-pids) và thư mục log riêng để khởi chạy thành công cụm 2 DataNode mô phỏng phân tán.
2. **Lỗi tràn đĩa tạm khi xử lý 25 triệu dòng (No space left on device):** Do WSL2 gán /tmp vào ramdisk chỉ có 699MB. Nhóm đã chuyển spark.local.dir sang /home/dell/spark_temp (nằm trên ổ đĩa cứng thực tế còn trống 940 GB) và loại bỏ thao tác dropDuplicates() để hạn chế tối đa việc shuffle ghi đĩa.
3. **Giới hạn bộ nhớ RAM 1.4 GB khi huấn luyện ALS:** Huấn luyện trên toàn bộ 25 triệu dòng gây sập máy ảo ngay lập tức. Nhóm áp dụng kỹ thuật lấy mẫu ngẫu nhiên đơn giản 3% (~750.000 dòng đánh giá) kết hợp cấu hình rank=10, maxIter=5 và thiết lập checkpoint, giúp Spark giải phóng bộ nhớ đệm trung gian định kỳ.
4. **Lỗi AMBIGUOUS_REFERENCE khi ghép bảng:** Cả 3 tệp movies, links và metadata đều chứa các cột có tên trùng nhau (title, movieId, id). Nhóm đã chủ động đổi tên cột tường minh (TenPhim, link_movieId, meta_id) trước khi thực hiện thao tác join.
5. **Dữ liệu văn bản đa dòng chứa ký tự ngắt dòng:** Tệp movies_metadata.csv chứa các đoạn tóm tắt nội dung dài có ký tự xuống dòng \n khiến Spark hiểu nhầm là bản ghi mới. Nhóm đã cấu hình các tùy chọn đọc: quote="", escape="", multiLine=True để phân tích cú pháp chuẩn xác.
6. **Xung đột phiên bản phần mềm:** Hadoop 3.3.6 yêu cầu Java 8 trong khi các bản PySpark mới yêu cầu Java 11/17. Nhóm đã lựa chọn chính xác phiên bản PySpark 3.5.1 tương thích hoàn toàn với Java 8. Đồng thời nâng cấp lên Airflow 3.3.2 để tương thích với môi trường Python 3.14 của Ubuntu 26.04.
7. **Mất trạng thái Kafka khi khởi động lại WSL:** Mỗi lần tắt máy, WSL xóa sạch dữ liệu trong /tmp khiến Kafka KRaft mất metadata cluster. Nhóm đã xây dựng kịch bản khởi động tự động: tạo lại cluster ID, định dạng storage log và tự động tái lập topic movie_logs.

4. Các phần chưa hoàn thành, hạn chế kỹ thuật và kế hoạch tiếp tục

Bên cạnh các kết quả tích cực đã đạt được, nhóm nhìn nhận khách quan các hạn chế kỹ thuật hiện tại cần được nâng cấp và hoàn thiện trong giai đoạn 2 của đồ án:

Các hạn chế kỹ thuật hiện tại của hệ thống:

1. **Quy mô dữ liệu huấn luyện ALS:** Mô hình hiện mới chỉ được huấn luyện trên mẫu 3% dữ liệu do rào cản tài nguyên RAM máy ảo, chưa tận dụng hết 100% sức mạnh của 25 triệu đánh giá lịch sử.

2. **Điểm số dự đoán ở tầng Gold chưa được Clipping:** Do bản chất của phép phân rã ma trận tuyến tính, một số điểm dự đoán vượt quá giới hạn 5.0 (ví dụ 7.10, 7.89), cần được bổ sung bước chuẩn hóa về thang điểm [0.5, 5.0].
3. **Chưa tích hợp dữ liệu streaming vào gợi ý:** Dữ liệu log thu nhận từ Kafka mới dừng ở mức lưu trữ xuống HDFS Silver, chưa được kết hợp với mô hình để phục vụ tính năng cập nhật gợi ý theo thời gian thực (Real-time Serving).
4. **Airflow DAG còn ở mức đơn nhiệm:** DAG hiện tại mới chỉ kích hoạt một task BashOperator đơn lẻ, chưa được chia nhỏ thành chuỗi quy trình đa nhiệm liên hoàn (Multi-task Pipeline) với các dependency chặt chẽ.
5. **Spark vẫn chạy ở chế độ local[*]:** Chưa triển khai cụm phân tán đa Worker nodes để khai thác toàn diện tính chất "Điện toán phân tán" ở tầng tính toán.
6. **Giao diện Web mới ở mức Dashboard tra cứu:** Chưa hoàn thiện tính năng người dùng tương tác trực tiếp (chọn phim yêu thích, chấm sao, nhận kết quả gợi ý cá nhân hóa ngay lập tức kèm ảnh Poster phim).

Kế hoạch hành động chi tiết triển khai giai đoạn 2:

Bảng 8. Kế hoạch chi tiết triển khai giai đoạn 2 hoàn thiện đồ án

STT	Công việc
1	Tối ưu mô hình ALS và chuẩn hóa điểm dự đoán
2	Tích hợp Kafka và Spark Streaming
3	Xây dựng Airflow Multi-task DAG
4	Triển khai Spark Cluster đa Worker
5	Nâng cấp Web, tích hợp TMDb và Dashboard
6	Benchmark, báo cáo và đóng gói mã nguồn

KẾT LUẬN

Đến thời điểm báo cáo tiến độ, nhóm đã hoàn thành toàn bộ các thành phần cốt lõi của một hệ thống Xử lý Dữ liệu lớn toàn diện theo mô hình Medallion Lakehouse:

1. Đã triển khai và vận hành ổn định cụm lưu trữ phân tán Apache Hadoop HDFS với 2 DataNode độc lập và hệ số nhân bản dữ liệu bằng 2.
2. Xây dựng thành công quy trình xử lý theo lô bằng PySpark, làm sạch và chuẩn hóa hơn 25 triệu bản ghi đánh giá và 62.000 bộ phim, chuyển đổi sang định dạng Snappy Parquet giúp giảm 69.8% không gian lưu trữ.
3. Huấn luyện thành công mô hình học máy Lọc cộng tác ALS của Spark MLlib, hoàn tất pipeline tính toán và sinh Top 5 gợi ý phim cá nhân hóa lưu trữ bền vững tại tầng Gold trên HDFS.
4. Triển khai hoàn chỉnh đường ống thu nhận luồng thời gian thực với Apache Kafka (chế độ KRaft) và PySpark Structured Streaming, hứng liên tục log tương tác và ghi micro-batch xuống HDFS.
5. Thiết lập quy trình tự động hóa điều phối bằng Apache Airflow DAG và xây dựng giao diện phục vụ dữ liệu trực quan bằng FastAPI Dashboard.

Toàn bộ hệ thống vận hành ổn định trên môi trường tài nguyên hạn chế nhờ các kỹ thuật tối ưu hóa bộ nhớ và phân vùng đĩa thực tế. Trong giai đoạn tiếp theo, nhóm sẽ tập trung nâng cao chất lượng mô hình ALS, mở rộng cụm tính toán Spark phân tán, hoàn thiện pipeline Airflow đa nhiệm và phát triển giao diện Web tương tác trực quan để hoàn thành đồ án một cách xuất sắc nhất.